63d23ca0-2897-45e4-8ac6-5dc0e2652a

<prev_user>

<request>

You are working on a task in the 'dkratzert/FinalCif' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/FinalCif/FinalCif (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-17T07:53:04.356Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Make a plan: The Cifcontainer class controls the ordering of ciif keys in the main table. I would like to bring all keys starting with _vrf to the top of the table and the saved cif. See if there is a function in gemmi to get all _vrf keys so that the search is fast.
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prev_assistant>

Now I have a complete picture. Here is my plan:

<plan>

Plan: Bring `_vrf_*` keys to the top of the main table and saved CIF

Background
- The key order in both the main table and the saved CIF is controlled by `CifContainer.order` (a list) and `CifContainer.order_cif_keys()`.
- `order_cif_keys()` iterates over `self.order` **in reverse** and calls `block.move_item(…, 0)` — so the *first* item in `self.order` ends up first in the block.
- The main table is populated from `cif.key_value_pairs()` → `keys_with_essentials()`, which iterates the block directly and returns two `sorted()` t (preserving document order) and a non-`_vrf_*` sublist (sorted alphabetically, as today).
- Return them concatenated: `[*vrf_questions, *sorted_non_vrf_questions]` and `[*vrf_with_values, *sorted_non_vrf_with_values]`.
- Then in `key_value_pairs()` the separator remains between the two blocks, but within each block `_vrf_*` keys come first.

3. (`Optional`) Persist the `_vrf` ordering through `cif.order`
In `AppWindow._fill_table()` (around line 2086), when a new key is inserted into `self.cif.order`, `_vrf_*` keys should be inserted at position 0 instead of at `row_num`. This ensures the order list stays in sync with the desired display order.

Files to change
- `finalcif/cif/cif_file_io.py` — `order_cif_keys()` and `keys_with_essentials()` (and optionally the order-sync block in `AppWindow._fill_table()`).
- `finalcif/appwindow.py` — the order-sync block inside the table-fill loop (optional but keeps everything consistent).

Tests to add / update
- Unit tests for `order_cif_keys()`: verify that in a CifContainer with mixed keys (including `_vrf_DIFF003_x`), after calling `order_cif_keys()` the first keys in `cif.keys()` are the `_vrf_*` ones.
- Unit tests for `keys_with_essentials()`: verify that `_vrf_*` entries appear before non-`_vrf_*` entries in both the questions and with-values sublists.
- Existing tests for key ordering should continue to pass.
</plan>

<current>

<request>

You are working on a task in the 'dkratzert/FinalCif' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/FinalCif/FinalCif (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-17T08:08:02.036Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Implement the plan:

Plan: Bring `_vrf_*` keys to the top of the main table and saved CIF

Background
- The key order in both the main table and the saved CIF is controlled by `CifContainer.order` (a list) and `CifContainer.order_cif_keys()`.
- `order_cif_keys()` iterates over `self.order` **in reverse** and calls `block.move_item(…, 0)` — so the *first* item in `self.order` ends up first in the block.
- The main table is populated from `cif.key_value_pairs()` → `keys_with_essentials()`, which iterates the block directly and returns two `sorted()` lists (questions + with_values).
- `_vrf_*` keys are currently sorted alphabetically together with everything else.

Gemmi API for finding `_vrf_*` keys
There is no dedicated prefix-filter API in gemmi; the fast approach is to iterate once over `block` pairs and collect keys that start with `_vrf`:
```python
vrf_keys = [item.pair[0] for item in self.block if item.pair and item.pair[0].startswith('_vrf')]
```
This is a single O(n) pass, which is already what `keys()`, `values()`, and `keys_with_essentials()` do.

Changes needed

1. `CifContainer.order_cif_keys()` in `cif_file_io.py`
After the existing ordering loop (which places the `order`-list keys at the front), add a second pass that collects all `_vrf_*` keys currently in the block and moves each one to position 0 (in reverse order, so the first `_vrf_` key ends up first). Because `move_item(index, 0)` is already used for normal ordering, the

same pattern works:
```python
# Move _vrf keys to top (collected first, then moved in reverse
so order is preserved)
vrf_keys = [item.pair[0] for item in block if item.pair and
item.pair[0].startswith('_vrf')]
for key in reversed(vrf_keys):
    try:
        block.move_item(block.get_index(key), 0)
    except (RuntimeError, ValueError):
        pass
```
This ensures all `_vrf_*` entries appear at the very top of the saved CIF file.

2. `CifContainer.keys_with_essentials()` in `cif_file_io.py`
This method returns the rows for the main table. Currently it returns two independently `sorted()` lists. Change the return so that `_vrf_*` keys float to the top:
- Partition `questions` and `with_values` each into a `_vrf_*` sublist (preserving document order) and a non-`_vrf_*` sublist (sorted alphabetically, as today).
- Return them concatenated: `[*vrf_questions, *sorted_non_vrf_questions]` and `[*vrf_with_values, *sorted_non_vrf_with_values]`.
- Then in `key_value_pairs()` the separator remains between the two blocks, but within each block `_vrf_*` keys come first.

3. (`Optional`) Persist the `_vrf` ordering through `cif.order`
In `AppWindow._fill_table()` (around line 2086), when a new key is inserted into `self.cif.order`, `_vrf_*` keys should be inserted at position 0 instead of at `row_num`. This ensures the order list stays in sync with the desired display order.

Files to change
- `finalcif/cif/cif_file_io.py` — `order_cif_keys()` and `keys_with_essentials()` (and optionally the order-sync block in `AppWindow._fill_table()`).
- `finalcif/appwindow.py` — the order-sync block inside the table-fill loop (optional but keeps everything consistent).

Tests to add / update
- Unit tests for `order_cif_keys()`: verify that in a CifContainer with mixed keys (including `_vrf_DIFF003_x`), after calling `order_cif_keys()` the first keys in `cif.keys()` are

the `_vrf_*` ones.
- Unit tests for `keys_with_essentials()`: verify that `_vrf_*` entries appear before non-`_vrf_*` entries in both the questions and with-values sublists.
- Existing tests for key ordering should continue to pass.
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.